VIETNAM NATIONAL UNIVERSITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications in Computer Vision

ASSIGNMENT A1

Instructor(s): Le Thanh Sach, Nguyen Quoc Minh

Author: Phung Bich Lan

Student ID: 2570438

HO CHI MINH, Nov, 2026

Source Code & Delivery package

A.1. Source repo: [CO5085_261_BTL_A1_PhungBichLan]

A.2. System environment

Parameter table	Detail configuration
Language & Library	Python 3.12.13 Pytorch Torchvision
Hardware	NVIDIA Google Colab
CUDA	13.2

A.3. Project directory structure

```
├── BTL_A1/
├──── source          // source code
├────── data          // data (train, validation, test)
├────── models        // model
├──── report          // report (.pdf)
├── BTL_A2/
├──── source          // source code
├────── data          // data (train, validation, test)
├────── models        // model
├──── report          // report (.pdf)
├── BTL_E1-E2-E3/
├──── data            // data (train, validation, test)
├────── E1
├──────── models      // model
├──────── .ipynb      // source code
├────── E2
├──────── models      // model
├──────── .ipynb      // source code
├────── E3
├──────── models      // model
├──────── .ipynb      // source code
├────── report        // report for E1, E2, E3
├── pages/            // .html of experiment
└── index.html        // main html of github page
```

A.4. Quick start commands

Open btl_a1.ipynb to see the training & testing result

Due to heavy size of models, reference result will upload to Google Drive then it accessible by email @hcmut.edu.vn : CO5085_261_BTL_A1_PhungBichLan

Project Execution / Timeline

Timeline	Task & Deliverables	Primary outputs
19/10/2026	E1-E3: GitHub pages	GitHub pages delivery for draft version
26/10/2026	A1.1: GitHub pages	GitHub pages deliver for Dataset + Protocol (data processing, training plan draft)
02/11/2026	E1, E2: source + report A2.1: GitHub pages	E1, E2: complete source and report A2.1: GitHub pages delivery for paper, dataset and metric
09/11/2026	E3: source + report A1.2: source + report	E3: complete source and report A1.2: complete source and report

Contents

Source Code & Delivery package 2
Project Execution / Timeline 4
List of Notations & Acronyms 7
Chapter 1. Introduction 8
1.1. Objectives 8
1.2. Scope of assignment 8
Chapter 2. General setup 9
2.1. Dataset & data preprocessing pipeline 9
2.1.1. Dataset selection & integration 9
2.1.2. Data preprocessing 10
2.1.3. Data loading & batching strategy 10
2.2. Common training setup 11
2.2.1. Cross Entropy Loss function 11
2.2.2. Optimizer and Scheduler 12
2.2.3. Hyperparameter 14
2.3. Common pipeline 14
2.3.1. Training pipeline 14
2.3.2. Evaluation pipeline 17
Chapter 3. Experiment A1 19
3.1. Model architecture 19
3.1.1. CNNs family – ResNet50 19
3.1.2. Transformer family – ViT-B/16 22
3.1.3. Model capacity summary 24
3.2. Experimental result and analysis 25
3.2.1. Overral performance comparison 25
3.2.2. Training and validation analysis with loss/accuracy curve 27
3.2.3. Sample error analysis 29
3.3. Discussion 30

References 32

List of Notations & Acronyms

CNNs	Convilutional Neural Networks
CE	Cross Entropy
SGD	Stochastic Gradient Descent
TP	True Positive
FP	False Positive
TN	True Neagtive
FN	False Negative
ViT	Vision Transformer

Chapter 1. Introduction

1.1. Objectives

The primary objective of this assignment is to systematically explore, implement and evaluate different deep learning fine-tuning strategies for large-scale image classification by using a shared dataset and a unified training setup (identical optimizer, scheduler and data processing pipeline). After completing the assignment, the following key expected outcomes include:

- Dataset Preparation and Training Pipeline: Understanding and implementation of Dataset/DataLoader, data augmentation, and a unified training and evaluation pipeline for large-scale image datasets.
- Fine-tuning Strategy Analysis: Understanding and comparison of different training strategies, including training from scratch vs. pretrained models, freezing the entire backbone, partially freezing the backbone, and full fine-tuning, using at least one CNN and one Transformer (ViT or its variants).
- Architecture and Computational Trade-off Analysis: Insights into how different architectures and fine-tuning strategies affect feature extraction, model capacity, parameter efficiency, computational costs (training time, inference time, and memory consumption), and classification accuracy.
- Experimental Evaluation: Well-founded assessment of model performance through evaluation metrics, loss-accuracy curves, confusion matrices, and misclassification analysis.

1.2. Scope of assignment

The scope of this assignment focuses on implementing and evaluating fine-tuning strategies for large-scale image classification using at least one CNN model and one Transformer model under a unified experimental setup.

- Model: CNNs, Transformer
- Fine-tine strategy: Explore and compare different training configurations:
  + Training from scratch.
  + Pretrained model with the entire backbone freeze.
  + Pretrained model with partially freeze full backbone layers.
  + Pretrained model with full fine-tuning.

Chapter 2. General setup

2.1. Dataset & data preprocessing pipeline

2.1.1. Dataset selection & integration

In assignment, experiment are conducted on the ImageNet-100 dataset which download via Huggingface open source (asafaa/imagent100 · Datasets at Hugging Face)

Dataset overview: 133982 RGB images (different resolution) across 100 classes (separate to 123982 training images, 5000 validation images and 5000 test images)

Dataset	Number of samples
Train set	123982
Validation set	5000
Test set	5000

The dataset exhibits a uniform class distribution, help to against class imbalance issues during model training phase.

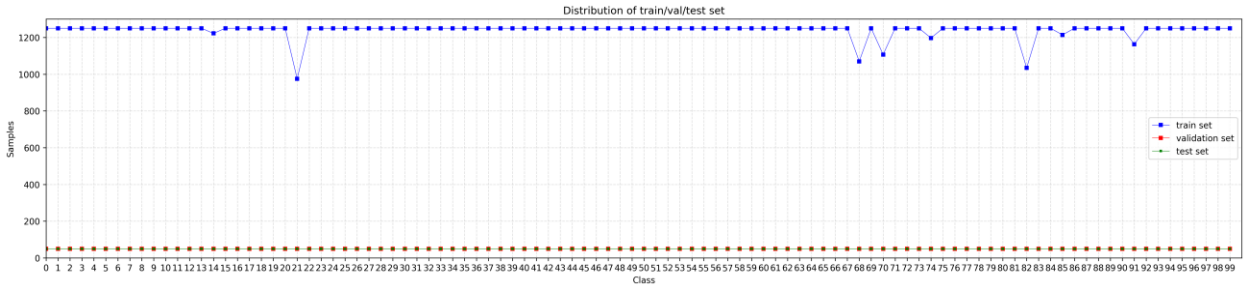


Figure 1. Class distribution for training/validation/testing set

This dataset is suitable for evaluating fine-tuning strategies for image classification for three main reasons:

- Sufficient Complexity: The dataset contains diverse visual patterns, textures, colors, and object structures, providing a suitable basis for evaluating how pretrained CNN and Transformer architectures adapt their learned feature representations to a target classification task.
- Dataset Size: The dataset provides a sufficient number of training samples to investigate the effects of different fine-tuning strategies, including freeze full backbone, partial fine-tuning, and full fine-tuning, while observing training dynamics and generalization performance without requiring high-end computing resources.
- Open Source and Accessibility: The dataset is publicly available and can be easily downloaded through PyTorch or other standard dataset repositories, facilitating

reproducible experiments and consistent comparisons across different model architectures and training configurations.

2.1.2. Data preprocessing

To ensure a consistent and fair evaluation across all experiments, all models share the same data processing pipeline implemented using torchvision.transforms:

- Normalization: apply ImageNet standard normalization using the predefined channel-wise mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225] to standardize input images.
- Data Augmentation (training set only): apply RandomResizedCrop(224), RandomHorizontalFlip(p=0.5), and ColorJitter(brightness=(0.8, 1.2), contrast=(0.8, 1.2)) to improve model generalization and reduce overfitting. This normalization value is same with ImageNet1k pretrained model
- Tensor Conversion: convert input images into PyTorch tensors using transforms.ToTensor() before normalization.
- Evaluation Pipeline: no data augmentation is applied to the validation and test sets to ensure consistent and deterministic model evaluation.

Note: ImageNet normalization is used to match the input preprocessing convention of pretrained CNN and Transformer models trained on ImageNet.

2.1.3. Data loading & batching strategy

The ImageNet-100 dataset is downloaded from Hugging Face and organized into separate training and validation sets. All images are extracted from ZIP archives, while the corresponding class labels are retrieved from Labels.json.

All dataset are wrapped in Pytorch `torchvision.datasets` and streamed using `torch.utils.data.DataLoader` to optimize memory consumption and GPU utilization during training loop:

- Dataset download: download the ImageNet-100 training and validation sets from Hugging Face using hf_hub_download. Each dataset split consists of five ZIP archives, which are extracted into their respective directories. Class labels are retrieved from the accompanying Labels.json file.
- Batch size: number of training samples processed before the model update its parameter. It strongly impact to computational efficiency, gradient estimation, memory usage and model generalization. A small batch size produces noiser gradient estimate due to calculate from fewer samples. But sometime, the noise help the

optimizer escape local minima or poor solution, but it also make the training unstable. Larger batch size provide smoother update.

- Shuffling: enabled for training set at each epoch to break sample correlation; disabled for validation and test sets to maintain evaluation consistency
- Multi-processing: config with `num_workers=8`, `pin_memory=True` to accelerate data transfer from CPU memory to GPU VRAM

2.2. Common training setup

2.2.1. Cross Entropy Loss function

In classification problems, model predicts the probability of each class for any given input. Due to each data point belongs to only one class, the expected output should be probability 1 for true class, 0 for others. Then, cross-entropy loss is a way to measure how close a model's predictions are to the expected result.

Apply cross entropy for multiclass classification, it introduces multiclass cross entropy loss: $CE = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C}\left(y_{i,j}\log\left(p_{i,j}\right)\right)$, where:

- N: number of samples
- C: number of classes
- $y_{i,j}$: 1 if class j is correct for sample i, 0 otherwise
- $y_{i,j}$: model-predicted probability of sample i being in class j

Interpretability for multiclass classification using CE loss function:

- Only predicted same as true label contributes towards the loss as for other labels being zero does not affect to loss computation.
- Heavily penalize the wrong class predict by model then the CE result becomes larger to indicate the high loss.

Module `torch.nn` provides essential loss function `nn.CrossEntropyLoss` for multi-class classification task which combines `LogSoftmax` on an input, followed by `NLLLoss` (negative log likelihood loss) [2]

To prove this, manually calculate `nn.LogSoftmax` and apply `nn.NLLLoss` on the output then compare with result output of `nn.CrossEntropyLoss` by following statement:

```
logits = torch.tensor([[2.0, 1.0, 0.1]])
label = torch.tensor([0])

ce_lossfunction = nn.CrossEntropyLoss()
ce_loss = ce_lf(logits, label)

logit_softmax = nn.LogSoftmax(dim=1)
```

```
# batch size = 1, classes = 3
# true class = 0

# LogSoftmax + NLLLoss
# loss calculate
```

```
nllloss = nn.NLLLoss()

logit_prob = logit_softmax(logits)                  # apply softmax to logits
logit_softmax_loss = nllloss(logit_prob, labels)    # apply NLLLoss

logit_loss = nllloss(logits, label)                 # apply only NLLLoss
```

```
…
Execute
…
```

```
using nn.CrossEntropyLoss : 0.417030
manually (LogSoftmax + nn.NLLLoss) : 0.417030
manually nn.NLLLoss : -2.000000
nn.CrossEntropyLoss is equivalent to manually (LogSoftmax + NLLLoss)
```

From result, we can directly using `nn.CrossEntropyLoss` apply to logit output from model without softmax non-linear activation in output layer.

2.2.2. Optimizer and Scheduler

Design the `modelOptimizer` class to manage the optimization process combine with a learning rate scheduler which play a crucial role in determining model convergence, stability and final performance.

Introducing Adam (Adaptive Moment Estimation) as an optimization algorithm that updates model parameters using the computed gradients of the loss function on mini-batches of training data. Adam combines the advantages of Momentum and RMSProp by maintaining two exponentially decaying moving averages: the first moment (mean of gradients) and the second moment (uncentered variance of gradients). These estimates allow Adam to adapt the learning rate for each parameter individually. The first moment helps smooth gradient updates and accelerate convergence, while the second moment adjusts the update magnitude based on gradient variability. Additionally, weight decay helps prevent excessively large weights and reduces overfitting.

`StepLR` scheduler introduce to decay learning rate (`lr`) of each parameter group by `gamma` every `step_size` epoch. Example for scheduler StepLR with `step_size=30`, `gamma=0.1` from initialize learning rate `lr=0.05`:

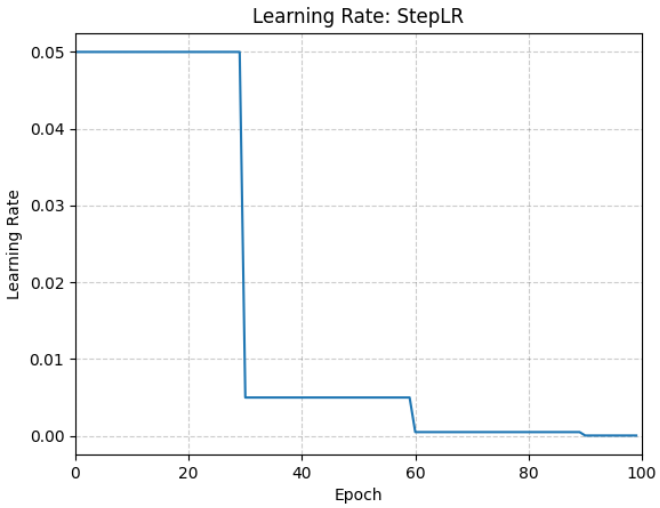


Figure 2. StepLE scheduler (lr=0.05, step_size=30, gamma=0.1)

Class definition with combine optimizer Adam and scheduler StepLE as following implementation:

```
class modelOpt_ADAM():
    def __init__(self, model, lr_bb=0.0001,
lr_head=0.001,weight_decay=0.001, betas=(0.9, 0.999), eps=1e-8,
model_type="resnet"):
        self.model = model
        self.lr_bb = lr_bb
        self.lr_head = lr_head
        self.weight_decay = weight_decay
        self.betas = betas
        self.eps = eps

        if model_type == "resnet":                          (1)
            head_params = list(map(id, self.model.fc.parameters()))
        elif model_type == "vit":
            head_params = list(map(id,
self.model.heads.head.parameters()))

        backbone_params = [p for p in self.model.parameters() if
id(p) not in head_params]                                   (2)

        self.optimizer = torch.optim.Adam(
            [
                {'params': backbone_params, 'lr': self.lr_bb},
                {'params': self.model.fc.parameters(), 'lr':
self.lr_head} if model_type == "resnet" else {'params':
self.model.heads.head.parameters(), 'lr': self.lr_head}
            ],
            weight_decay=self.weight_decay,
            betas=self.betas,
            eps=self.eps)                                   (3)
```

```
(1) head classifier
parameter
(2) backbone classifier
(3) Adam optimizer with
specific hyperparameters
(4) zero gradient
```

`def zero_grad(self):`
`  self.optimizer.zero_grad(set_to_none=True)  (4)`
 	
`def step(self):`
`  self.optimizer.step()`
 	
`def get_lr(self):`
`  return {`
`  'lr_bb': self.optimizer.param_groups[0]['lr'],`
`  'lr_head': self.optimizer.param_groups[1]['lr']`
`  }`	

`class modelScheduler_StepLR():`
`def __init__(self, optimizer, step_size=10, gamma=0.1):`
`  self.optimizer = optimizer`
`  self.step_size = step_size`
`  self.gamma = gamma`
`  self.scheduler =`
`torch.optim.lr_scheduler.StepLR(self.optimizer,`
`step_size=self.step_size, gamma=self.gamma)  (5)`
 	
`def scheduler_step(self):`
`  self.scheduler.step()`
	(5) step scheduler with specific hyperparameters

2.2.3. Hyperparameter

Hyperparameter	Value	Explain
batch_size	64	Total training sample > 123982, batch size 64 => 1937 batchs in epoch Batch size = 64 is balance with computation efficiency with time and memory
lr_head	0.001	Choose a relatively small learning rate to fine-tine head classifier only
Lr_bb	0.0005	Choose a relatively small learning rate to fine-tine backbone
mm	0.9	Accelerates convergence by incorporating information from previous gradients and helps reduce oscillations during optimization
weight_decay	0.0005	Applies L2-style regularization to discourage excessively large weights and help reduce overfitting
num_epochs	20	for both strategy with pretrained models
step_size	10	Step size to reduce lr_bb and lr_head
gamma	0.2	Multiplies the learning rate by 0.2 whenever the scheduler reaches the specified step size
num_worker	4	Uses 4 subprocesses for data loading to enable parallel data preparation and reduce data-loading overhead
input_shape	224*224	Input image shape of models

2.3. Common pipeline

2.3.1. Training pipeline

Image classification pipeline defines the complete workflow of data processing, model training, validation, and final testing. It begins with loading and preprocessing the dataset, follow by feeding training batches into the model to generate predictions and calculate the loss. The model parameters are then updated through backpropagation with

apply an optimization algorithm and learning rate scheduler also consider. After each epoch, the model is evaluated on the validation set to monitor performance and select the best-performing checkpoint. Finally, the selected best model is evaluated on the independent test set to measure its generalization performance.

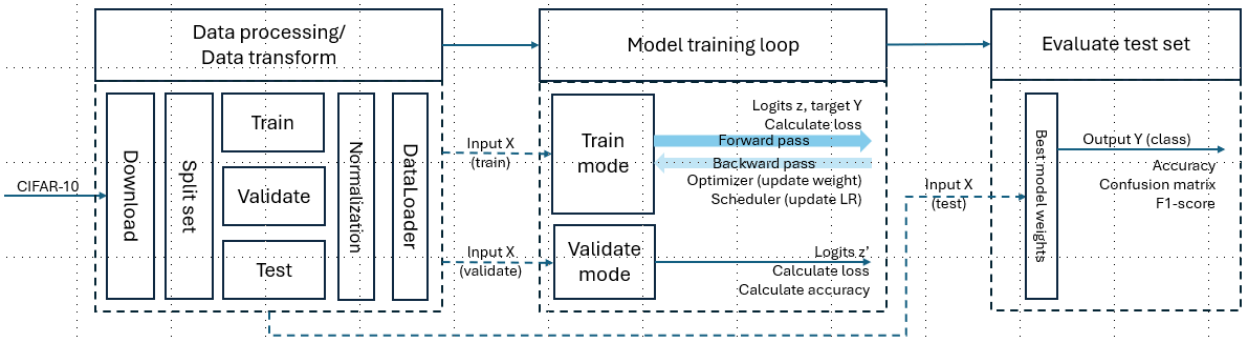


Figure 3. Image classification pipeline

The implementation covers the complete image classification workflow, from data preparation and model training to validation, best-model selection, and final testing. The common implementation consists of the following components, which are shared across all models. The specific models are described separately.

`path = hf_hub_download(` (1)
`  repo_id=repo_id,`
`  filename=filename,`
`  repo_type="dataset"`
`)`
 	
`with zipfile.ZipFile(path, "r") as z:`
`  z.extractall(datapath + "/" + split)` (2)
	[Download]
	Download dataset ImageNet-100 from HuggingFace
	Unzip dataset
	
	(train and validation only)

`helper_move_dataset(datapath_train, datapath_test, total2move=50)`	[Split set] Move 50 per class of training set to test set (use to verify generalization performance of model) Total testset = 50*100 = 5000

`transforms.Compose([`
`  transforms.RandomResizedCrop(input_shape),`
`  transforms.RandomHorizontalFlip(),`
`  transforms.ColorJitter(brightness=0.2, contrast=0.2),`
`  transforms.ToTensor(),`
`  transforms.Normalize(mean=rn50_ft_norm_mean, std=rn50_ft_norm_std)`
`])`
	[Normalization]
	Normalization input to shape (224, 224) from original shape. Then random horizontal flip and apply data augmentation with normalize input value same with pretrained model

`DataLoader(dataset, batch_size, shuffle=True, num_workers= num_workers, pin_memory=True)`	[DataLoader] Use PyTorch DataLoader to load the training, validation, and test dataset in mini-batch.

	Enable shuffle the training data at each epoch to improve training efficiency, while keeping validation and test data in a fixed order for consistent evaluation. Use multiple worker processes (num_workers) to load data in parallel. Enable pin_memory to potentially accelerate data transfer from CPU to GPU.
nn.CrossEntropyLoss() : Using CrossEntropy loss function for multiple class classification torch.optim.Adam : design an optimizer with learning rate, (betas, eps) and weigh_decay from Hyperparameter torch.optim.lr_scheduler.StepLR : design StepLR scheduler with step_size and gamma from Hyperparameter	[Common training setup] Ensure all common training setup are ready
Specific model architecture design (model) model_rn50 = models.resnet50(weights=models.ResNet50_Weights.DEFAULT) model_vitb16 = models.vit_b_16(weights=models.ViT_B_16_Weights.DEFAULT)	[Model creation] This part should be specific for each model
[Train mode] for each epoch: model.train() (1) for each mini-batch: output_z = model(input_x) (2) model_optimizer.zero_grad (3) loss backward (4) model_optimizer.step (5) calculate loss/accuracy of training set model_optimizer.scheduler_step (6) update global loss min, global accuracy max [Evaluate mode] for each epoch: model.eval() (7) with torch.no_grad(), for each mini-batch (8) output_z = model(input_x) (9) calculate loss/accuracy of validate set update global loss min, global accuracy max [Save checkpoint] for each epoch: save last checkpoint save best model check point (10)	[Model training loop] (1) Enable training mode (2) output_z from model (3) reset gradients of all parameter to 0 (4) calculate gradient (5) update weights (6) update LR (7) Enable evaluate mode (8) Disabling gradient calculation (9) output_z from model (10) save best model which testing on evaluate test provide the best accuracy

[Save time] Time of training loop	
best_model.load() best_model.eval() (1) with torch.no_grad(), for each mini-batch (2) output_z = best_model(input_x) (3) calculate loss/accuracy of test set save prediction results	[Evaluate test set] (1) Enable evaluate mode (2) Disabling gradient calculation (3) output_z from best_model
To evaluate model generalization: + Loss/Accuracy – epoch (train set) + Loss/Accuracy – epoch (validate set) + Loss/Accuracy on test set Confusion matrix	[Evaluation test set]

2.3.2. Evaluation pipeline

To evaluate the performance and generalization ability of the image classification model, a consistent evaluation pipeline is applied to all 3 models.

- Loss and accuracy

Training set: monitor loss/accuracy value after each epoch training to evaluate the model's learning progress. If loss value does not decrease significant, or the accuracy remains low, the model may be underfitting, the learning rate may be inappropriate, or the model may require further optimization. However, very high training accuracy or very low loss does not necessarily indicate the good generalization, model may be overfitting on the data training. Consequently, the model is highly confident when using its training data, but it performs poorly when evaluated on other datasets.

Validation set: evaluate after each training epoch using a separate validation dataset that is not used to update the model parameters. The purpose is to measure how well the model generalizes to unseen data during training. A decreasing validation loss and increasing validation accuracy indicate improved generalization. If training loss continues to decrease while validation loss increases, the model may be overfitting the training data. Comparing training and validation metrics helps identify overfitting or underfitting and select an appropriate model checkpoint.

Test set: evaluated on test set after training with best model (which release the best accuracy on validation set) to measure the final classification performance on unseen data.

So, the model visualizes a goof generalization when both training and validation loss low and close to each other, while training and validation accuracy are both high and similar. This indicates that the model learn effectively and generalizes well to unseen data. Finally, perform evaluate on test set expects the low loss and high accuracy.

- Confusion Matrix and Precision, Recall, and F1-score

It uses to analyze classification results for each class, showing correct predictions and misclassifications. It helps identify frequently confused classes and differences in classification performance across classes. These observations may indicate potential class imbalance, insufficient training samples, or similarities between classes.

From matrix results, further analysis and appropriate adjustments, such as collecting more representative data, applying class weighting, or using data augmentation, can be performed to improve the classification performance.

With predicted class and known correct class, confusion matrix introduces 4 important evaluate metrics:

- TN (True Negative): the number of samples that are correctly predicted as negative (predicted class = Y''), meaning the actual class is negative (correct class = Y'')
- TP (True Positive): The number of samples that are correctly predicted as positive (predicted class = Y'), meaning the actual class is positive (correct class = Y')
- FN (False Negative): The number of samples that are incorrectly predicted as negative (predicted class =Y''), meaning the actual class is positive (correct class = Y')
- FP (False Positive): The number of samples that are incorrectly predicted as positive (predicted class = Y'), meaning the actual class is negative (correct class = Y'')

Confusion Matrix		Predicted Class		
		1	2	3
Actual Class	1	TN	FP	TN
	2	FN	TP	FN
	3	TN	FP	TN

Figure 4. Confusion matrix of multiple-class classification (class = 3)

Beside these confusion matrix metrics, Precision, Recall and F1-score provide a more detailed evaluation of class-wise performance which calculate from TN, TP, FN, FP.

- Precision: Measures the percentage of correctly predicted positive samples out of the total number of samples predicted as positive. A higher precision indicates a lower number of False Positives (FP)

$$Precision = \frac{TP}{TP + FP}$$

- Recall: Measures the percentage of correctly predicted positive samples out of the total number of positive samples. A higher recall indicates fewer False Negatives (FN)

$$Recall = \frac{TP}{TP + FN}$$

- F1-score: The harmonic mean of precision and recall, provide a balanced measure of both metrics. A higher F1-score indicates a better balance between precision and recall.

$$F1 - score = \frac{2}{\frac{1}{Percision} + \frac{1}{Recall}}$$

+ High Precision, Low Recall: the model makes a few positive errors (FP), but it misses many actual positive samples (FN). Its positive predictions seems reliable, but it fails to identify many positive samples (FN)

+ Low Percision, High Recall: the model identifies most actual positive samples (TP) by visualize fewer FN, but it produces many false positive samples (FP). The model capture most potive samples, however, its positive predictions are less reliable

+ High Precision, High Recall (near 1): the model identifies many actual positive samples (few FN) and correctly identifies most positive samples while making a few false positive error (few FP). This result indicates the strong classification performance

+ Low Percision, Low Recall (near 0): the model produces many false positive and false negative errors, indicating poor performance in identifying the positive class.

In conclusion, the model demonstrates strong performance when it achieves a good balance between Precision and Recall, correctly identifying most positive samples while making few false positive and false negative errors.

Chapter 3. Experiment A1

3.1. Model architecture

3.1.1. CNNs family – ResNet50

Development from CNN, ResNet50 has a well-organized structure that makes model possible to go deep without losing important information.

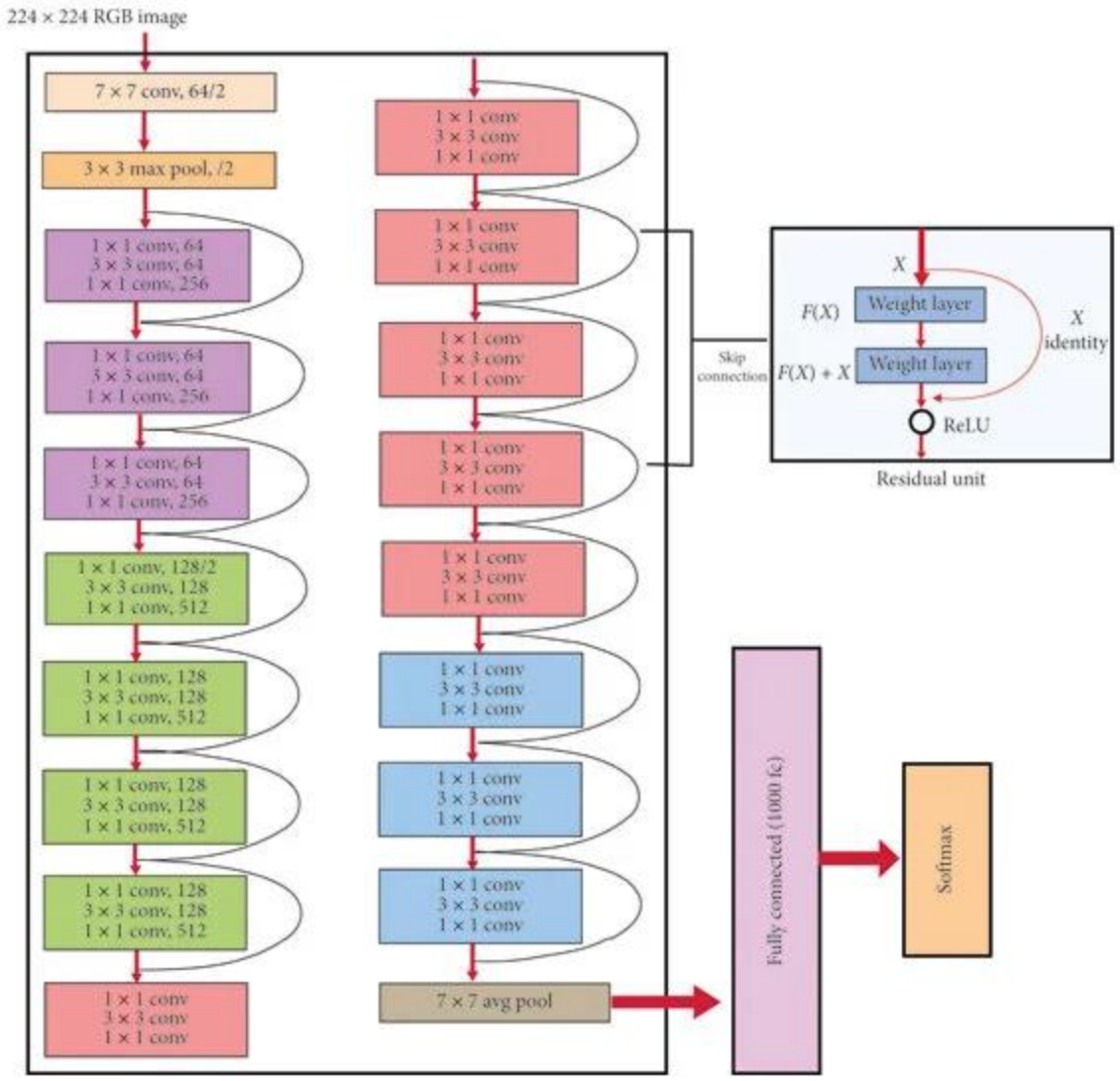


Figure 5. ResNet50 model architecture (CNN family)

ResNet50 – Residual network with 50 layers, is a deep CNN architecture which introduces residual learning through shortcut connections, allowing information and gradients to propagate more effectively through the network.

The model architecture consists of:

Component	Configuration	Explain
Input	224x224x3	RGB image input
Conv1	7x7, 64 filters, stride 2	Extract initial visual features
MaxPool	3x3, stride 2	Reduce spatial dimensions
Conv2_x	3 Bottleneck blocks	Extract low-level features
Conv3_x	4 Bottleneck blocks	Extract intermediate features
Conv4_x	6 Bottleneck blocks	Extract high-level features
Conv5_x	3 Bottleneck blocks	Extract semantic features

Global Average Pooling	1×1	Aggregate spatial features
Fully Connected	2048 → num_classes	Produce class predictions

Component Conv1 (convolution kernel 7x7) extract initial low-level visual features and simple patterns. Batch normalization apply to normalize intermediate feature activations to improve training stability and convergence. Non-linear function ReLU allows network to learn more complex feature representations. After that, apply Max pooling layer to reduce spatial dimension while retaining important activation responses, decrease computational cost in subsequent layers.

Conv2 to conv5 (residual stage) consists of 16 bottleneck block in total. Each stage progressively extracts more complex and abstract visual representations. Bottleneck block is the fundamental building unit of ResNet50. Each block contains three convolutional layers and one shortcut connection.

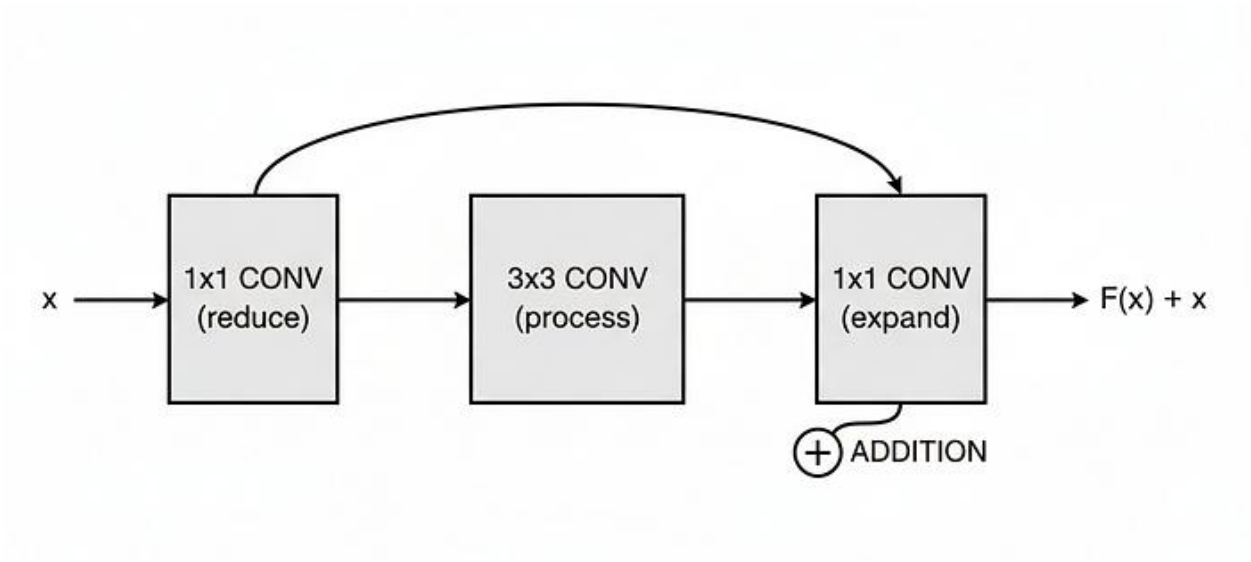


Figure 6. Bottleneck block in ResNet50

- The first 1x1 convolution reduces the number of channels, making subsequent operations computationally cheaper.
- The 3x3 convolution extracts spatial features from neighboring pixels.
- The final 1x1 convolution expands the feature channels to provide a richer representation.

The main important contributions of ResNet is the residual connection. Instead of directly learning a mapping H(x), the network learns a residual function by add directly input to output. It is helpful to improve gradient propagation during backpropagation and preserve

informations from previous layers. Thereby, model allows deeper networks to learn residual transformations instead of reconstructing the entire input representation

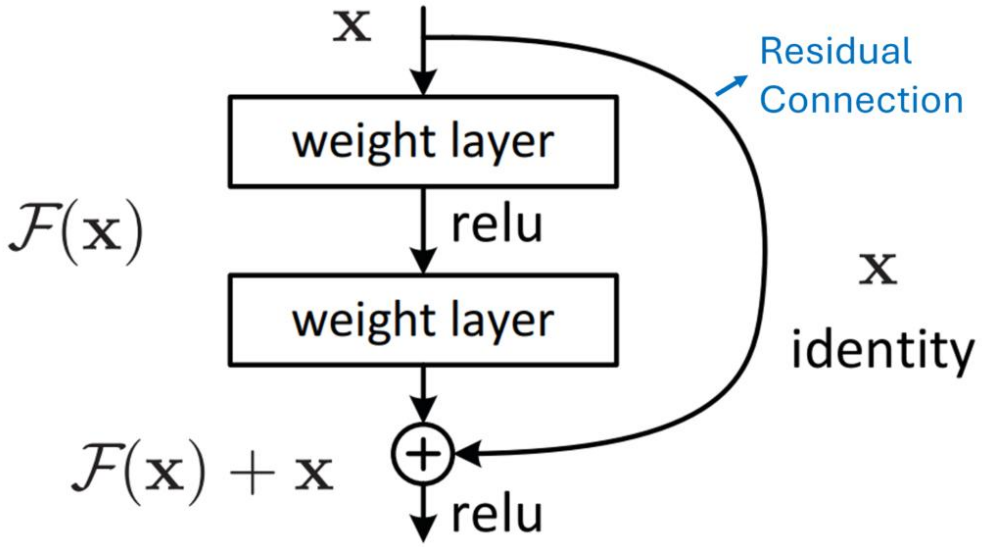


Figure 7. Residual connection

Global Average Pooling (GAP) calculates the average value of each feature channel across its entire spatial dimension. The final feature vector is passed into a fully connected layer to map extracted features to the target number of classes and generate classification logits.

Since the pretrained ResNet50 was originally trained on ImageNet-1K, its final classification layer must be replaced to match the 100 target classes.

3.1.2. Transformer family – ViT-B/16

Vision Transformer Base/16 (ViT-B/16) is a vision transformer architecture that apply the transformer encoder mechanism, originally developed for Natural Language Processing (NLP), to image classification tasks. Unlike CNN architectures such as ResNet50, which extract spatial features through convolutional filters, ViT divides an input image into fixed-size patches and processes them as a sequence of visual tokens using Self-Attention mechanisms.

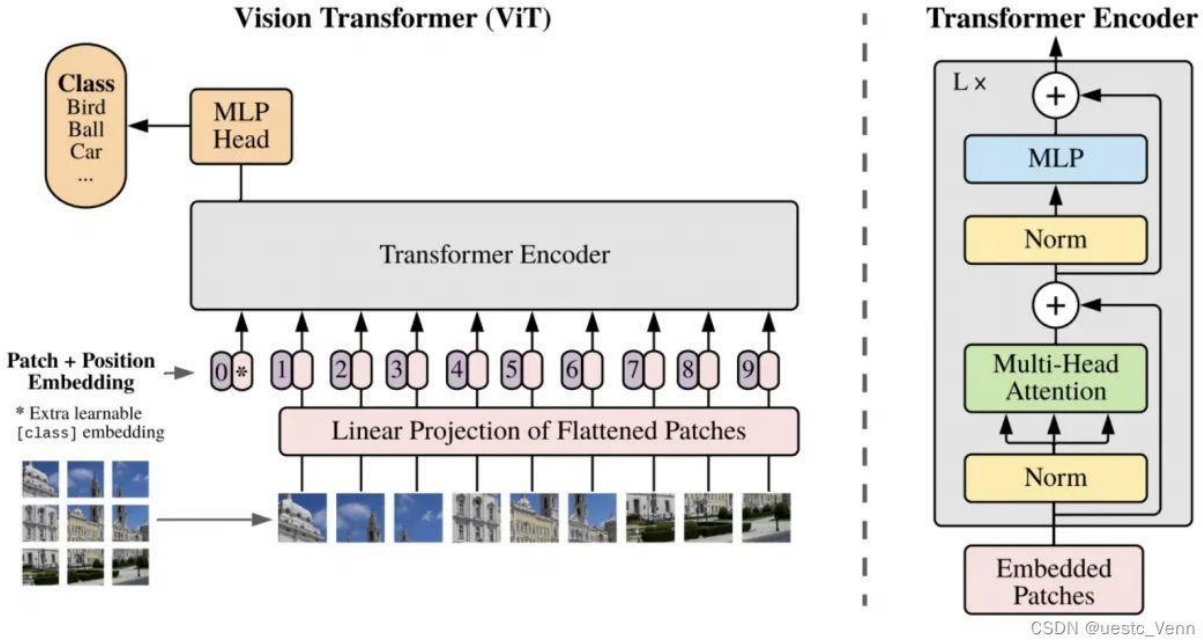


Figure 8. ViT-B/16 model architecture (Transformer family)

Component	Configuration	Explain
Input	224x224x3	RGB image input
Patch Size	16x16	Divide image into patches
Number of Patches	196	Generate visual tokens
Patch Embedding	768 dimensions	Convert patches into vectors
Class Token	1x768	Aggregate global image representation
Positional Embedding	197x768	Preserve patch positions
Transformer Encoder	12 layers	Extract global feature relationships
Hidden Dimension	768	Feature representation size
Attention Heads	12	Multi-head feature interaction
MLP Dimension	3072	Non-linear feature transformation
Classification Head	768 -> 100	Predict ImageNet-100 classes

ViT architecture includes the following major components:

- Patch + Position Embedding + Flatten: ViT-B/16 divides the input image into patches of size 16x16 pixels. The patches are flattened and projected into 768-dimensional embedding vectors using a learnable linear projection.
- Class Token [CLS]: model introduces a learnable classification token, which is prepended to the sequence of image patch embeddings. After adding CLS, resulting sequence contains 197 tokens, each with 768 dimensions.
- Transformer Encoder: introduces as a main feature extraction component of model. It consists of 12 identical Transformer Encoder block, each containing:

  - Layer Normalization: normalize token representations
  - Multi-Head Self-Attention: allows the model to learn relationships between different image patches
  - Residual connection: first residual connection combines the original input with the Self-Attention output
  - Layer Normalization: normalize token representations
  - Multi-Layer Perceptron: each token passes independently through a two-layer feed-forward network
  - Residual connection: second residual connection combines the MLP output with the previous representation
- Classification head: extracted from the Transformer output and passed into a fully connected layer. For this assignment, ViT-B/16 uses ImageNet-1K pretrained weights and is adapted to classify ImageNet-100 images

3.1.3. Model capacity summary

Model capacity of both models is evaluated by computing their numbers of trainable parameter.

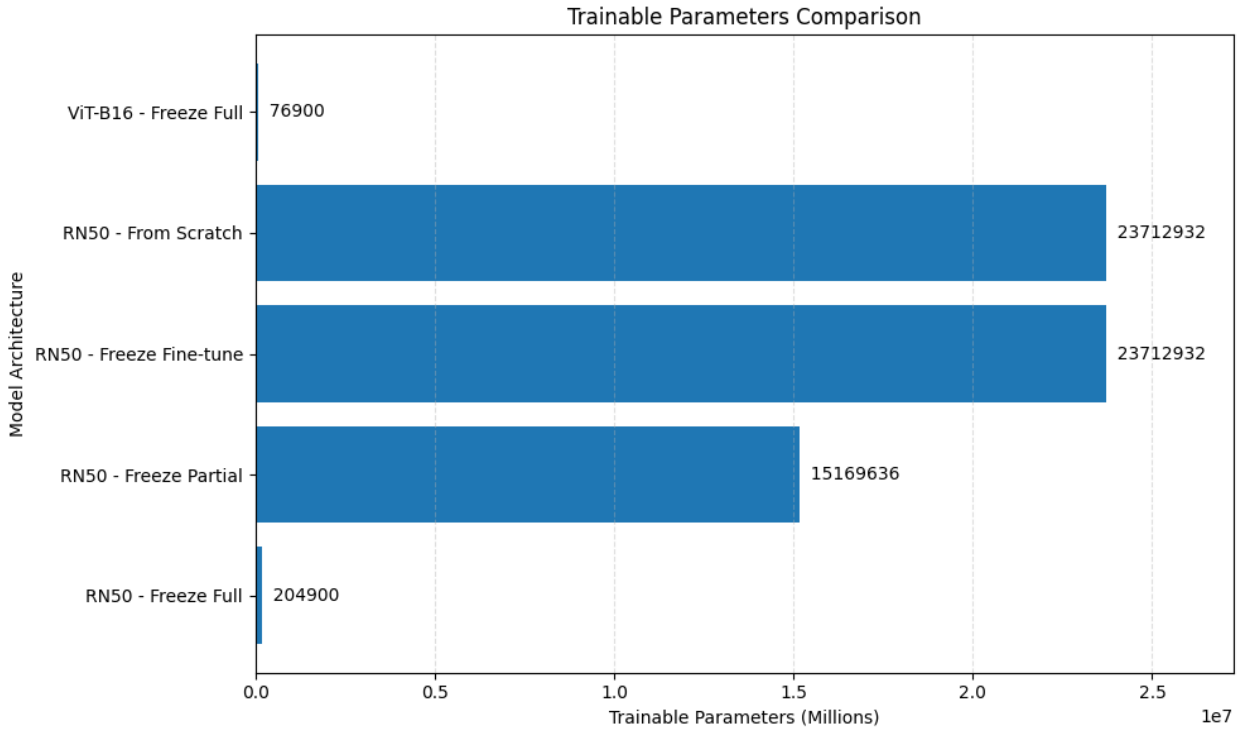


Figure 9. Model capacity of models

Model capacity is determined by the total number of parameters, while trainable capacity depends on the fine-tuning strategy. ViT-B/16 has a larger model capacity than ResNet50, but its Freeze full configuration updates only a small portion of the model.

3.2. Experimental result and analysis

3.2.1. Overral performance comparison

This experiment evaluates five model configurations:

- ResNet50 (Freeze Full Backbone): Freezes all pretrained backbone layers and trains only the classification head.
- ResNet50 (Partial Freeze): Freezes early layers while fine-tuning deeper layers and the classification head.
- ResNet50 (Full Fine-tuning): Fine-tunes all pretrained backbone parameters and the classification head.
- ResNet50 (From-Scratch): Trains the entire model from randomly initialized weights without pretrained knowledge.
- ViT-B/16 (Freeze Full Backbone): Uses pretrained Transformer weights, freezes the entire backbone, and trains only the classification head.

These five configurations allow comparison of different transfer learning strategies within ResNet50 and an alternative Transformer architecture (ViT-B/16), particularly in terms of classification performance and computational cost. The results obtained after training both models on the same datasets are presented below; the best model was selected based on the highest accuracy on the validation set.

	ResNet50				ViT-B/16
Metrics	FreezeFull	Partial	Full fine-tune	Scratch	FreezeFull
Train loss	0.3326	0.487556	0.88847	0.89166	0.164
Validation loss	0.4631	0.5838	0.99	0.99877	0.39538
Test loss	0.3356	0.516	0.9275	0.99	0.178
Train accuracy (%)	91.868	87.5734	76.832731	76.79	95.864
Validate accuracy (%)	88.46	84.88	74.68	74.3	90.7
Test accuracy (%)	91.76	86.7	75.74	74.1	95.86
Marco Precision	0.92	0.8728	0.766	0.752	0.95867
Marco Recall	0.9176	0.867	0.7574	0.741	0.9578
Marco F1-score	0.9179	0.86787	0.758	0.7415	0.95787
Total parameters	23712932	23712932	23712932	23712932	85875556
Trainable parameters	204900	15169636	23712932	23712932	76900
Training time (s)	3912.8	4032.17	9529.75	21202.36	13570.0

Testing time (s)	22.14	21.4	21.551	61.231	350.65

❖ Classification performance & analysis

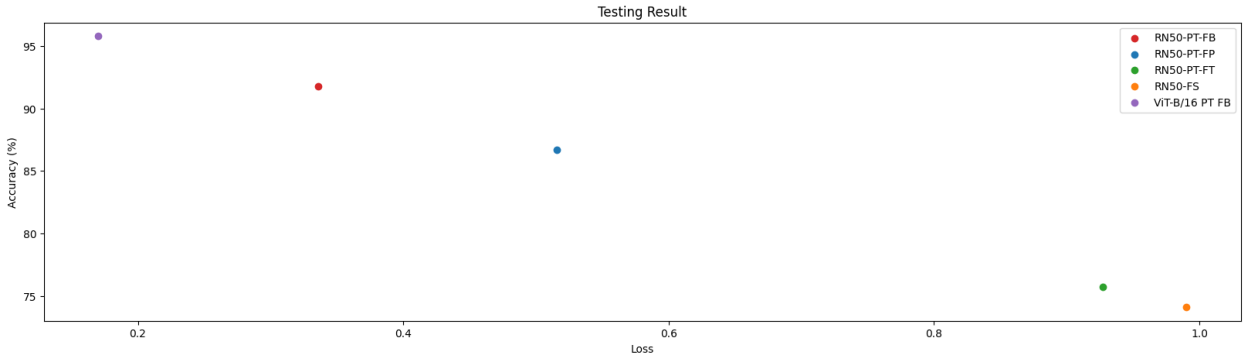


Figure 10. Loss and accuracy result on test set

The experimental results show clear differences among the four ResNet50 training strategies. ResNet50 with a freeze full backbone achieves the best classification performance among the ResNet50 configurations, reaching 91.76% test accuracy and average F1-score of 0.9179. This indicates that the pretrained ImageNet features provide strong representations for the target dataset, even when only the classification head is updated.

Partial fine-tuning achieves 86.7% test accuracy and F1-score of 0.86787, which are lower than those of the freeze full backbone strategy. Although more backbone parameters are updated, this does not necessarily improve performance. Fine-tuning deeper layers may alter useful pretrained representations and require more careful optimization.

Full fine-tuning obtains 75.74% test accuracy and F1-score of 0.758, showing a considerable performance decrease compared with the 2 previous strategies. This suggests that updating all pretrained parameters may lead to overfitting or damage useful pretrained features under the current dataset size and training configuration.

ResNet50 trained from scratch achieves 74.1% test accuracy and a Macro F1-score of 0.7415. This configuration show the lowest result of generation. This highlights the benefit of transferring knowledge from ImageNet pretraining rather than learning all visual representations from random initialization.

Overall, the results suggest that freezing the pretrained backbone is the most effective ResNet50 strategy in this experiment. Updating more parameters does not automatically lead to better generalization, particularly when the available training data and computational resources are limited.

Compare to best ResNet50 configuration, model ViT-B/16 achieves the highest classification performance, with 95.86% test accuracy and F1-score of 0.95787. Therefore, ViT-B/16 outperforms the best ResNet50 configuration. This suggests that the Transformer-based architecture can extract more discriminative representations for the ImageNet100 classification task under the experimental settings.

❖ Model capacity and training time

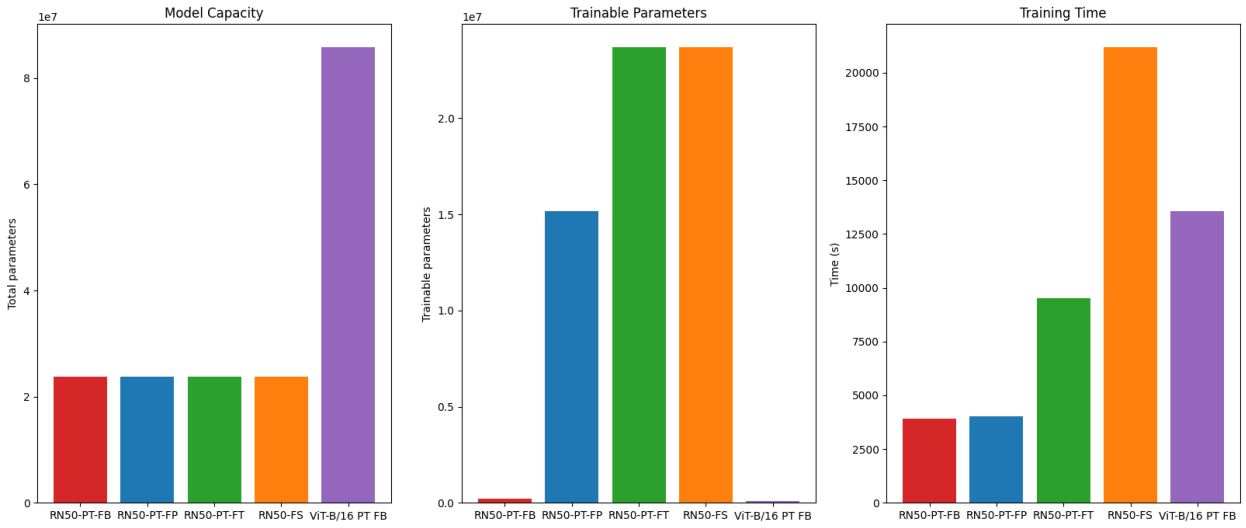


Figure 11. Model capacity and training time

All 4 ResNet50 configurations have the same total model capacity of 23.71 million parameters. However, their trainable parameter counts differ depending on the selected training strategy. In comparison, ViT-B/16 has a substantially larger model capacity, with approximately 85.88 million parameters, which is around 3.62 times that of ResNet50. Despite its larger capacity, the Freeze full ViT-B/16 configuration updates only 76900 parameters (approximately 0.09% of the total parameters), fewer than the 204900 trainable parameters in ResNet50 freeze full backbone.

These results indicate that ViT-B/16 offers higher classification performance but comes with greater model complexity and computational cost. Therefore, although ViT-B/16 achieves the best accuracy, Freeze full ResNet50 remains a more computationally efficient alternative, providing a reasonable balance between classification performance, model size, and training time.

3.2.2. Training and validation analysis with loss/accuracy curve

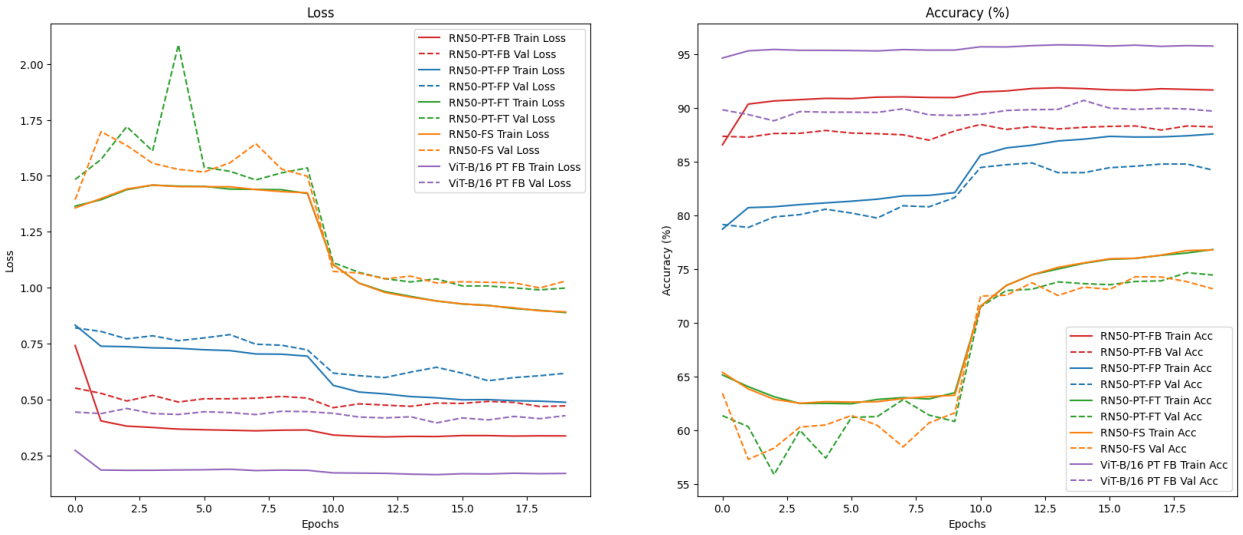


Figure 12. Loss/Accuracy curve of training/validation set

The final training and validation metrics provide additional insights into the learning and generalization behavior of each ResNet50 strategy.

- Freeze full backbone: it achieves the lowest training loss (0.3326) and validation loss (0.4631) among the ResNet50 configurations, together with the highest validation accuracy (88.46%). The relatively small difference between training and validation performance suggests good generalization.
- Partial fine-tune: it records a training loss of 0.4876 and validation loss of 0.5838, with validation accuracy of 84.88%. Its performance remains relatively stable but is lower than that of the freeze full backbone strategy.
- Full fine-tune: it shows higher training and validation losses of 0.8885 and 0.99, respectively. Its validation accuracy decreases to 74.68%, indicating that updating all pretrained parameters does not improve learning performance under the current experimental configuration.
- From-scratch: it obtains training and validation losses of 0.8917 and 0.9988, respectively, with 74.30% validation accuracy. This suggests that learning visual representations from random initialization is more challenging than transferring pretrained features.

Comparing the loss and accuracy results, ViT-B/16 (freeze full backbone) demonstrates better training and generalization performance than all 4 ResNet50 configurations. It achieves the lowest training loss (0.164), validation loss (0.3954), and test loss (0.178). ViT-B/16 has a relatively small gap between training and validation accuracy (5.16%), indicating a reasonable level of generalization. Freeze full backbone ResNet50 has

a gap of 3.408%. Although the gaps are relatively close, ViT-B/16 achieves higher absolute accuracy on both datasets.

3.2.3. Sample error analysis

Sample misclassifier that all models predict:



Figure 13. Error analysis of misclassifier image (target class: "eft")

RN50 Frozen: alligator lizard (88.79%) RN50 Partial: alligator lizard (21.89%) RN50 Full FT: stone wall (57.98%) RN50 Scratch: stone wall (41.36%) ViT-B/16: alligator lizard (58.84%)

For the "eft" class, all five models incorrectly classified the input image, but they produced different predicted classes and confidence scores. This confusion is particularly understandable because "eft" and "lizard" share several external visual characteristics, even though they belong to different animal groups: salamanders are amphibians, whereas lizards are reptiles. Consequently, the models may rely heavily on superficial visual similarities rather than learning sufficiently discriminative features to distinguish these categories.

Overall, this example highlights a fine-grained visual recognition challenge: the similarity between efts and lizards can lead models to confuse two biologically distinct animal groups. It also demonstrates that high confidence does not necessarily correspond to correct classification.

3.3. Discussion

The experimental results demonstrate that model performance depends not only on the architecture but also on the fine-tuning strategy and computational resources available. The comparison between ResNet50 configurations and ViT-B/16 provides insights into when transfer learning, fine-tuning, and different architectures should be preferred.

❖ When should fine-tuning be applied?

Fine-tune is useful when a pretrained model needs to adapt its learned representations to a target dataset. The appropriate strategy depends on the similarity between the source dataset (ImageNet1k) and the target dataset (ImageNet100), as well as the amount of available training data and computational resources.

- Freeze full backbone: Suitable when the target dataset is relatively similar to the pretraining dataset, the dataset is small, or computational resources are limited. Only the classification head is trained, preserving pretrained feature representations while reducing training cost. In this experiment, ResNet50 Freeze full achieved substantially better performance than the other ResNet50 fine-tuning configurations.
- Partial fine-tune: Suitable when pretrained features are generally useful but higher-level representations need adaptation. Unfreezing selected deeper layers allows the model to learn more task-specific features while retaining lower-level pretrained representations. However, this strategy requires more computation and may not always outperform a fully Freeze full backbone.
- Full fine-tune: Appropriate when the target dataset differs considerably from the source dataset, sufficient training data is available, and computational resources are adequate. Updating all backbone parameters provides greater flexibility to adapt the entire feature extractor. However, it also increases training time and the risk of overfitting or losing useful pretrained representations, particularly when the target dataset is limited.
- Training from scratch: More appropriate when pretrained weights are unavailable, the target domain is substantially different, or sufficient data and computational resources are available to learn useful representations from the beginning. In this experiment, ResNet50 trained from scratch achieved lower accuracy than its

pretrained counterparts, demonstrating the value of transfer learning under the given dataset and training conditions.

❖ When should ViT-B/16 be preferred over ResNet50?

ViT-B/16 is preferable when classification accuracy is a priority and sufficient computational resources are available. Unlike ResNet50, which uses convolutional operations to extract local spatial features, ViT-B/16 divides an image into patches and applies self-attention to capture relationships between different image regions.

However, ViT-B/16 also introduces several trade-offs:

- Higher model capacity: ViT-B/16 contains approximately 85.88 million parameters, compared with 23.71 million for ResNet50, making it substantially larger.
- Higher computational cost: Despite freezing its backbone and updating only 76900 parameters, ViT-B/16 required approximately 13570 seconds for training, compared with 3913 seconds for ResNet50 freeze full backbone.
- Slower inference: ViT-B/16 required approximately 350.64 seconds for testing, compared with 22.14 seconds for ResNet50 freeze full backbone.
- Global feature relationships: Self-attention enables ViT-B/16 to capture relationships across image patches, potentially benefiting classification tasks where global visual context is important. However, it also introduces greater computational requirements.

Further experiments on larger and more diverse datasets, different fine-tuning configurations, and computational environments are needed to validate these findings and establish more generalizable conclusions.

References

[1] What Is Cross-Entropy Loss Function? - GeeksforGeeks

[2] Introduction To Adam Optimizer - GeeksforGeeks

[3] Stochastic Gradient Descent with Momentum is Algorithmically Stable

[4] Frontiers | Integrating and optimizing genomic, weather, and secondary trait data for multiclass classification

[5] Evaluation Metrics in Machine Learning - GeeksforGeeks

[6] F1 Score in Machine Learning - GeeksforGeeks

[7] Vision Transformers (ViT) Explained | Pinecone

[8] MLIR Part 7 - Transformers

[9] Regularization Techniques in Machine Learning - GeeksforGeeks

[10] Multi-Head Attention Mechanism - GeeksforGeeks

[11] Vision Transformers vs. Convolutional Neural Networks (CNNs) - GeeksforGeeks

[12] Limitations of Single Attention Head

[13] Intro to Transformers: The Encoder Block | Edlitera

[14] How LLMs See Images, Audio, and More

[15] ResNet50 Explained: Architecture & When to Use It

[16] The Annotated ResNet50 | Towards Data Science

[17] ResNet50 Explained Step by Step: The Easiest Guide to Deep Residual Networks | by KararHaiderDeepVision | Medium

[18] Residual Connections in Machine Learning - ML Digest